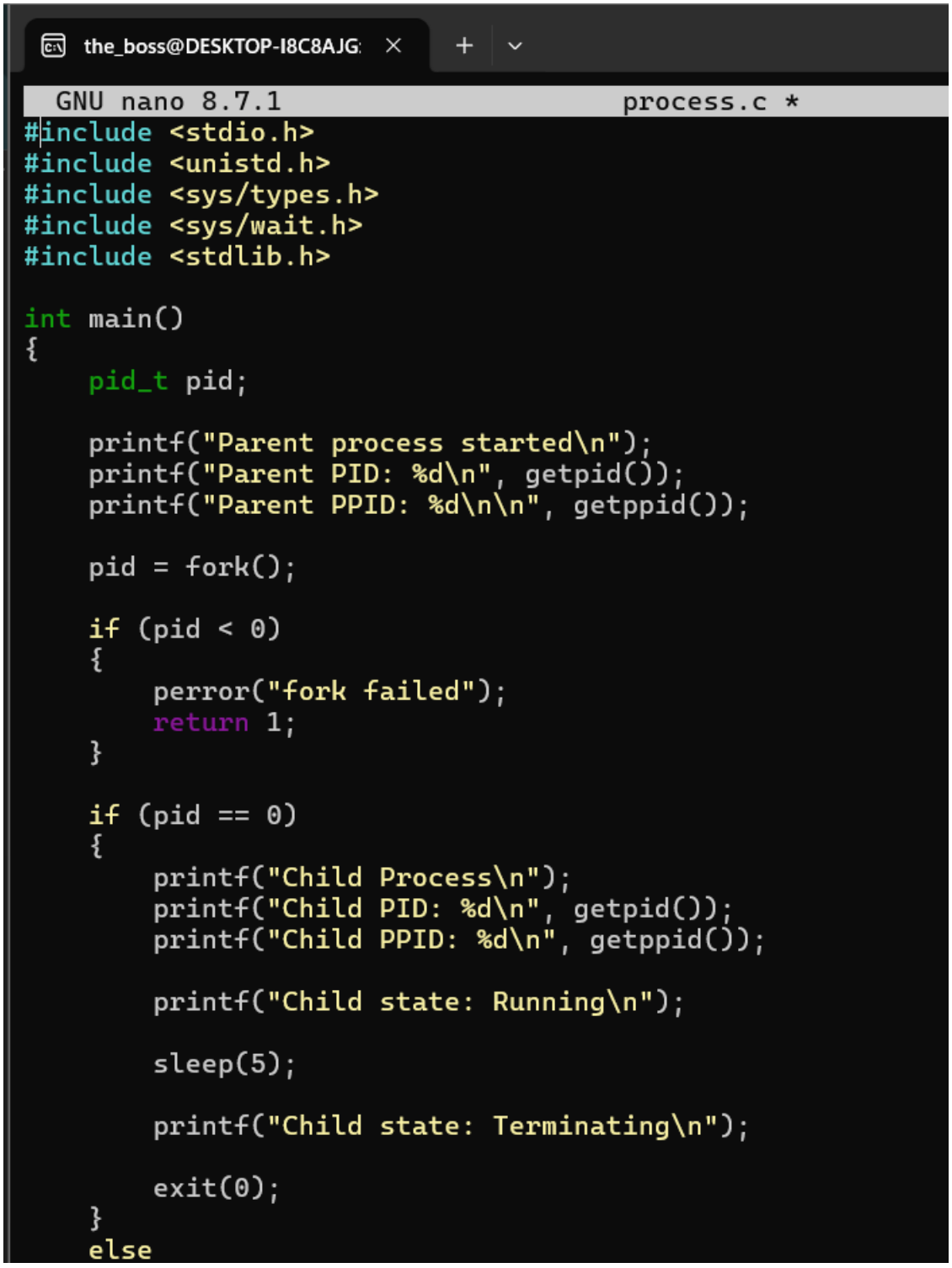
WEEK-3 Practical

Task-1 Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process States at different stages of execution



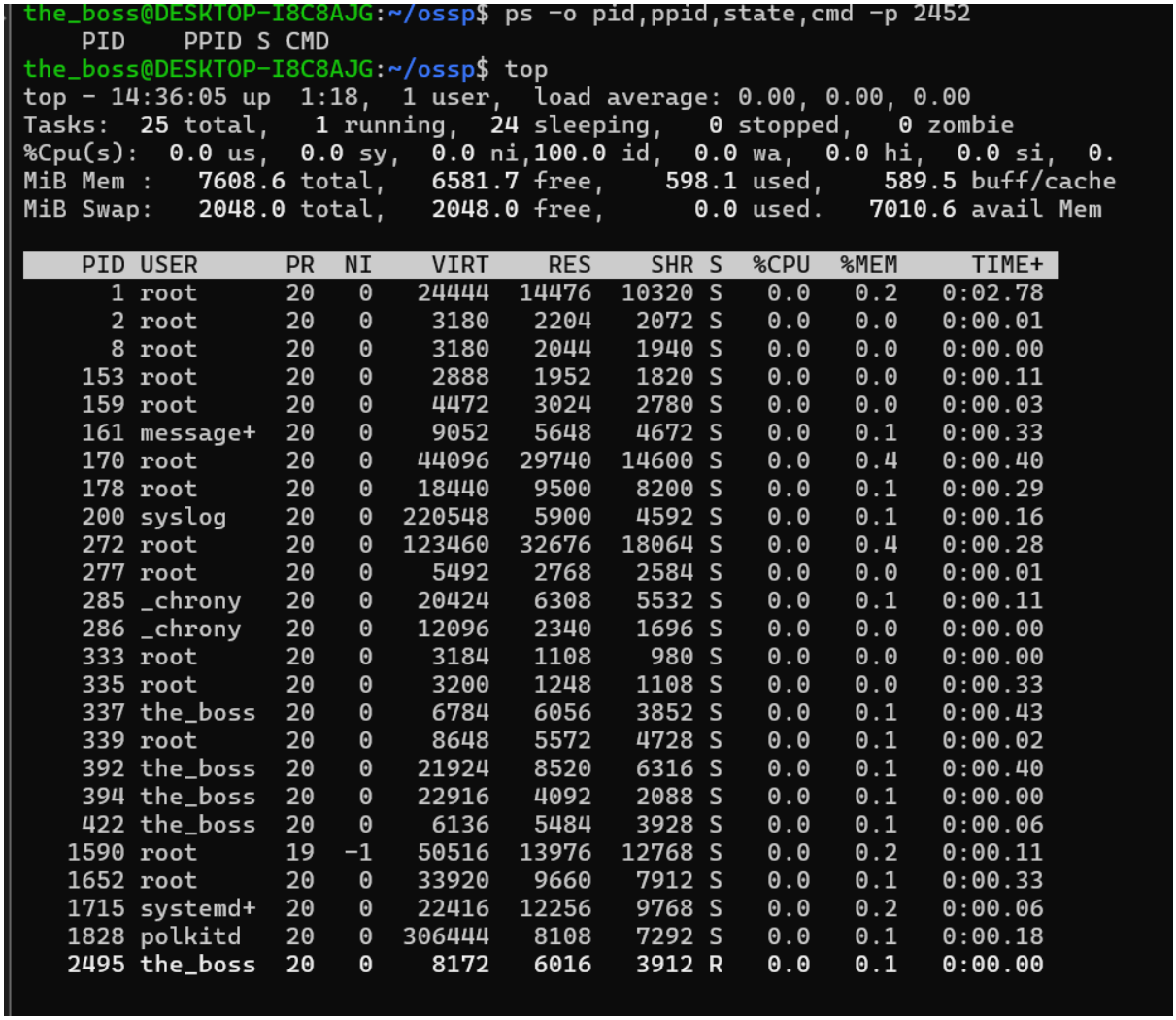
Output:

```
the_boss@DESKTOP-I8C8AJG:~/ossp$ nano process.c
the_boss@DESKTOP-I8C8AJG:~/ossp$ gcc -Wall -Wextra process.c -o bin/proces
the_boss@DESKTOP-I8C8AJG:~/ossp$ ./bin/process
Parent process started
Parent PID: 2416
Parent PPID: 337

Parent Process
Parent PID: 2416
Child PID: 2417
Parent state: Waiting for child
Child Process
Child PID: 2417
Child PPID: 2416
Child state: Running
Child state: Terminating
Parent state: Running again
Child process terminated
```

Report: I developed a C program using the fork() system call to create a parent process and a child process. The program displayed the PID and PPID of both processes and showed their states at different stages of execution. The parent process waited for the child using wait(), while the child process performed its task and then terminated. The experiment helped me understand the relationship between parent and child processes.

Task-2 : Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as PS, top, and /proc. Document the observations.



```
the_boss@DESKTOP-I8C8AJG:~/ossp$ ps -o pid,ppid,state,cmd -p 2452
    PID    PPID S CMD
the_boss@DESKTOP-I8C8AJG:~/ossp$ top
top - 14:36:05 up  1:18,  1 user,  load average: 0.00, 0.00, 0.00
Tasks:  25 total,   1 running,  24 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.
MiB Mem :   7608.6 total,   6581.7 free,    598.1 used,    589.5 buff/cache
MiB Swap:   2048.0 total,   2048.0 free,      0.0 used.   7010.6 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+
      1 root      20   0   24444  14476  10320 S   0.0   0.2   0:02.78
      2 root      20   0    3180   2204   2072 S   0.0   0.0   0:00.01
      8 root      20   0    3180   2044   1940 S   0.0   0.0   0:00.00
    153 root      20   0    2888   1952   1820 S   0.0   0.0   0:00.11
    159 root      20   0    4472   3024   2780 S   0.0   0.0   0:00.03
    161 message+  20   0    9052   5648   4672 S   0.0   0.1   0:00.33
    170 root      20   0   44096  29740  14600 S   0.0   0.4   0:00.40
    178 root      20   0   18440   9500   8200 S   0.0   0.1   0:00.29
    200 syslog    20   0  220548   5900   4592 S   0.0   0.1   0:00.16
    272 root      20   0  123460  32676  18064 S   0.0   0.4   0:00.28
    277 root      20   0    5492   2768   2584 S   0.0   0.0   0:00.01
    285 _chrony   20   0   20424   6308   5532 S   0.0   0.1   0:00.11
    286 _chrony   20   0   12096   2340   1696 S   0.0   0.0   0:00.00
    333 root      20   0    3184   1108    980 S   0.0   0.0   0:00.00
    335 root      20   0    3200   1248   1108 S   0.0   0.0   0:00.33
    337 the_boss  20   0    6784   6056   3852 S   0.0   0.1   0:00.43
    339 root      20   0    8648   5572   4728 S   0.0   0.1   0:00.02
    392 the_boss  20   0   21924   8520   6316 S   0.0   0.1   0:00.40
    394 the_boss  20   0   22916   4092   2088 S   0.0   0.1   0:00.00
    422 the_boss  20   0    6136   5484   3928 S   0.0   0.1   0:00.06
   1590 root      19  -1   50516  13976  12768 S   0.0   0.2   0:00.11
   1652 root      20   0   33920   9660   7912 S   0.0   0.1   0:00.33
   1715 systemd+  20   0   22416  12256   9768 S   0.0   0.2   0:00.06
   1828 polkitd   20   0  306444   8108   7292 S   0.0   0.1   0:00.18
   2495 the_boss  20   0    8172   6016   3912 R   0.0   0.1   0:00.00
```

Report: I designed an experiment to observe process state transitions using Linux tools such as ps, top, and the /proc filesystem. I observed the process while it was running, waiting, and terminating. The ps command displayed the PID, PPID, and process state, while top provided real-time process information. The /proc/<PID>/status file was used to examine detailed process information. The experiment helped me understand how Linux manages and monitors different process states.